

多功能算术/逻辑运算单元 (ALU)

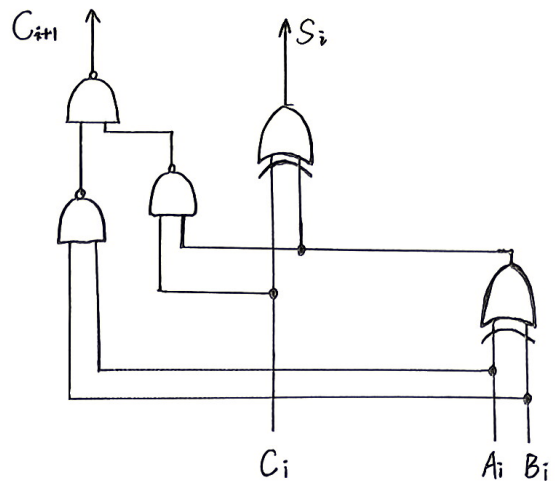
1) 基本思想

一位全加器 (FA) 的逻辑表达式为:

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = \overline{A_i B_i} \cdot \overline{C_i (A_i \oplus B_i)}$$

$$= A_i B_i + B_i C_i + C_i A_i$$



将全加器的功能扩展以完成多种算术逻辑运算.

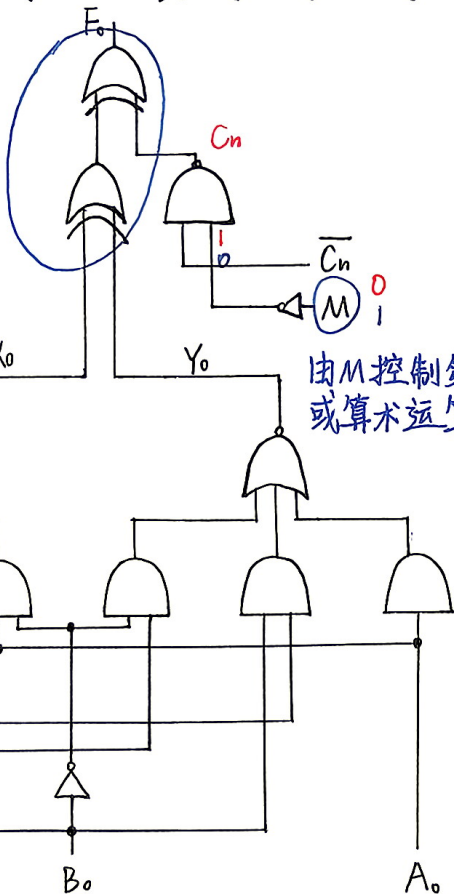
解决方案: 对 A, B 做预变换

① 将 A_i 与 B_i 先组合成由控制参数 S_0, S_1, S_2, S_3 控制的组合函数 X_i 和 Y_i .

② 再将 X_i, Y_i 与下一位进位数通过全加器进行全加.

这样一来, 不同的控制参数可以得到不同的组合函数, 因而能够实现多种算术运算和逻辑运算

核心部分是由
两级半加器构成的
全加器



进位信号

若 $M=0$ $\overline{Y}C_n$ 可通过与非门参与第2级半加器的运算.
可认为是算术运算.

若 $M=1$ C_n 无法通过与非门, 运算结果与 C_n 无关
可认为是逻辑运算.

整个电路可完成32种运算: 16种算术运算, 16种逻辑运算.

由M控制第二级半加器选择逻辑运算
或算术运算(所需的低位进位 C_n)

参数 S_0, S_1, S_2, S_3 分别控制输入 A_i 和 B_i ,
产生 Y 和 X 的函数.

其中:

Y_i 是受 S_0, S_1 控制的 A_i, B_i 的组合函数

X_i 是受 S_2, S_3 控制的 A_i, B_i 的组合函数

$$X_i = \overline{S_2} \overline{S_3} + \overline{S_2} S_3 (\overline{A_i} + \overline{B_i}) + S_2 \overline{S_3} (\overline{A_i} + B_i) + S_2 S_3 \overline{A_i}$$

$$Y_i = \overline{S_0} \overline{S_1} \overline{A_i} + \overline{S_0} S_1 \overline{A_i} B_i + S_0 \overline{S_1} \overline{A_i} \overline{B_i}$$

进一步化简: ALU的一位逻辑表达式

2个1位 $\rightarrow$ $X_i = S_3 A_i B_i + S_2 A_i \bar{B}_i$
 的
 二进制 $\rightarrow$ $Y_i = A_i + S_0 B_i + S_1 \bar{B}_i$

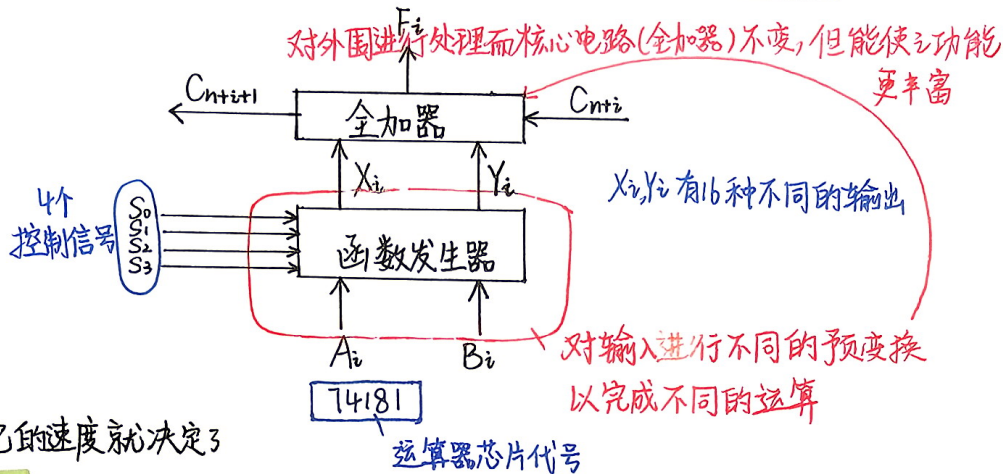
预变换电路的输出
 是电路原始输入 $A_i, B_i, S_0 \sim S_3$ 的函数.

$$X_i Y_i = S_3 A_i B_i + S_2 A_i \bar{B}_i \cdot A_i + S_0 B_i + S_1 \bar{B}_i = Y_i$$

$S_0 S_1$	Y_i	$S_2 S_3$	X_i
0 0	$\bar{A}_i$	0 0	1
0 1	$\bar{A}_i B_i$	0 1	$\bar{A}_i + \bar{B}_i$
1 0	$\bar{A}_i \bar{B}_i$	1 0	$\bar{A}_i + B_i$
1 1	0	1 1	$\bar{A}_i$

本位结果 $F_i = Y_i \oplus X_i \oplus C_{n+i}$

向高位进位 $C_{n+i+1} = Y_i + X_i C_{n+i}$



ALU是计算机中运算器最核心的部件, 它的速度就决定了计算机的运算速度, 故不能采用串行进位.

4位ALU的进位关系及逻辑电路

4位之间采用先行进位(并行进位)公式

根据 $C_{n+i+1} = Y_i + X_i C_{n+i}$, 每一位的进位公式可递推如下:

第0位向第1位的进位公式为:

$$C_{n+1} = Y_0 + X_0 C_n \quad (C_n \text{ 是向第0位(末位)的进位})$$

第1位向第2位的进位公式为:

$$C_{n+2} = Y_1 + X_1 C_{n+1} = Y_1 + Y_0 X_1 + X_0 X_1 C_n \rightarrow \text{可以不依赖于第0位向第1位的进位而同时进行}$$

第2位向第3位的进位公式为:

$$C_{n+3} = Y_2 + X_2 C_{n+2} = Y_2 + Y_1 X_2 + Y_0 X_1 X_2 + X_0 X_1 X_2 C_n$$

第3位的进位输出(即整个4位运算进位输出)公式为:

$$C_{n+4} = Y_3 + X_3 C_{n+3} = Y_3 + Y_2 X_3 + Y_1 X_2 X_3 + Y_0 X_1 X_2 X_3 + X_0 X_1 X_2 X_3 C_n$$

最后进位输出

逻辑表达式表明, 这是一个先行进位逻辑.

即第0位的进位输入 C_n 可以直接传送到最高位上去, 因而可以实现高速运算

进位产生函数 (进位发生输出) 令 $G = Y_3 + Y_2 X_3 + Y_1 X_2 X_3 + Y_0 X_1 X_2 X_3$

进位传递函数 (进位传递输出) 令 $P = X_0 X_1 X_2 X_3$. (引出G, P两个端子)

则 $C_{n+4} = G + P C_n$

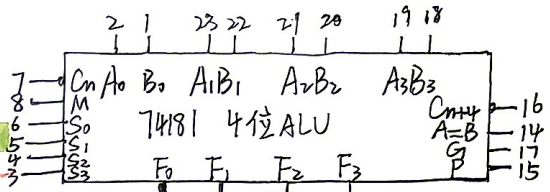
同时进行,
同时产生.

要增加额外
的硬件电路

多加这两个进位输出的目的是为便于实现多片(组)ALU之间的先行进位.

正逻辑表示的74181

向高位的进位 C_{n+4} 与本位和同时产生



高电平: 1
低电平: 0

$A=B$ 判断输入两数是否相等.

$3T+3T=6T$
 C_{n+4}

一路形成G,P,再形成高位的进位

两条路并行

先并行进位

C_{n+4} 不依赖于右

这4个运算位的

任何一个进位

为进一步级联

所有 $X_i Y_i$ 同时产生

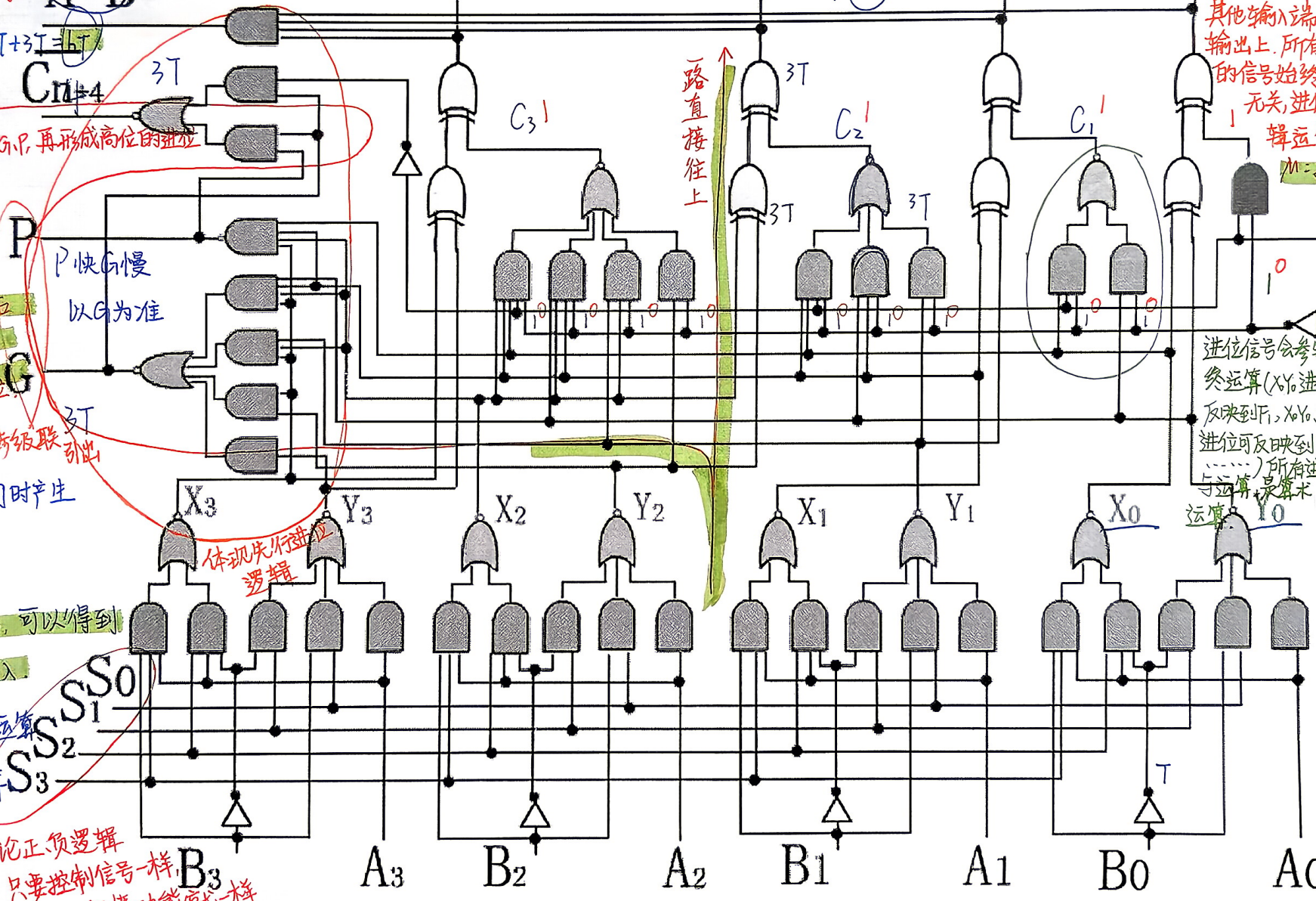
通过改变 $S_0 \sim S_3$, 可以得到

16个不同的输入

实现16种算术运算

16种逻辑运算

不论正负逻辑
只要控制信号一样
运算功能就一样

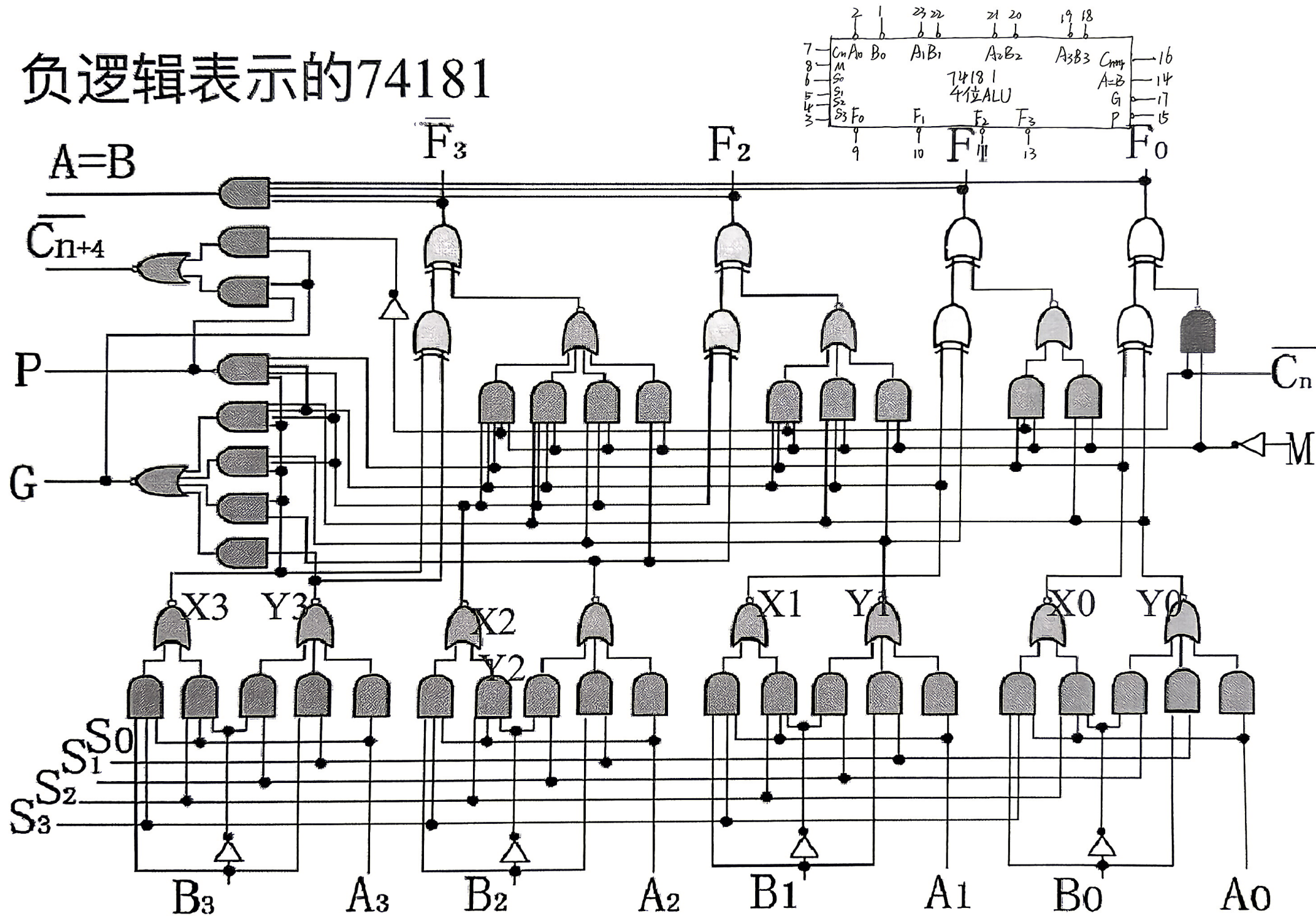


把所有与门封闭, 与门其他输入端的逻辑不会反映在输出上. 所有输入到第2级半加器的信号始终是1, 与另外的输入无关, 进位不参与运算, 是逻辑运算
M=控制是算术运算 or 逻辑运算

把所有与门打开, 与门其他输入端的逻辑都会反映在输出上. 进位信号会参与最终运算 ($X_i Y_i$ 进位可反映到 F_i , $X_i Y_i Y_i$ 与门输出上, 进位可反映到 F_{i+1} 上. 所有进位参与运算, 是算术运算. $T+3T=4T$.

2个4位二进制数同时输入预变换电路

负逻辑表示的74181



74181的逻辑功能表(查表即可)

注意:

① H: 高电平 L: 低电平

② *表示每1位均移到下个更高位, 即 $A^* = 2A$

③ 算术运算操作是用补码表示法来表示的, 其中"加"指算术加, 运算时考虑进位.

④ 减法是用补码方法进行的, 其中数的反码由内部产生, 而结果输出" A 减 B 减1", 因此做减法时要在最末位产生一个强迫进位(加1), 以产生" A 减 B "的结果.

⑤ " $A=B$ "输出端可指示两个数是否相等.

3) 并行加法器的进位逻辑

74181 ALU 是4位并行加法器, 如何组成16位的并行加法器?

4片(组) 74181 连接, 如何连?

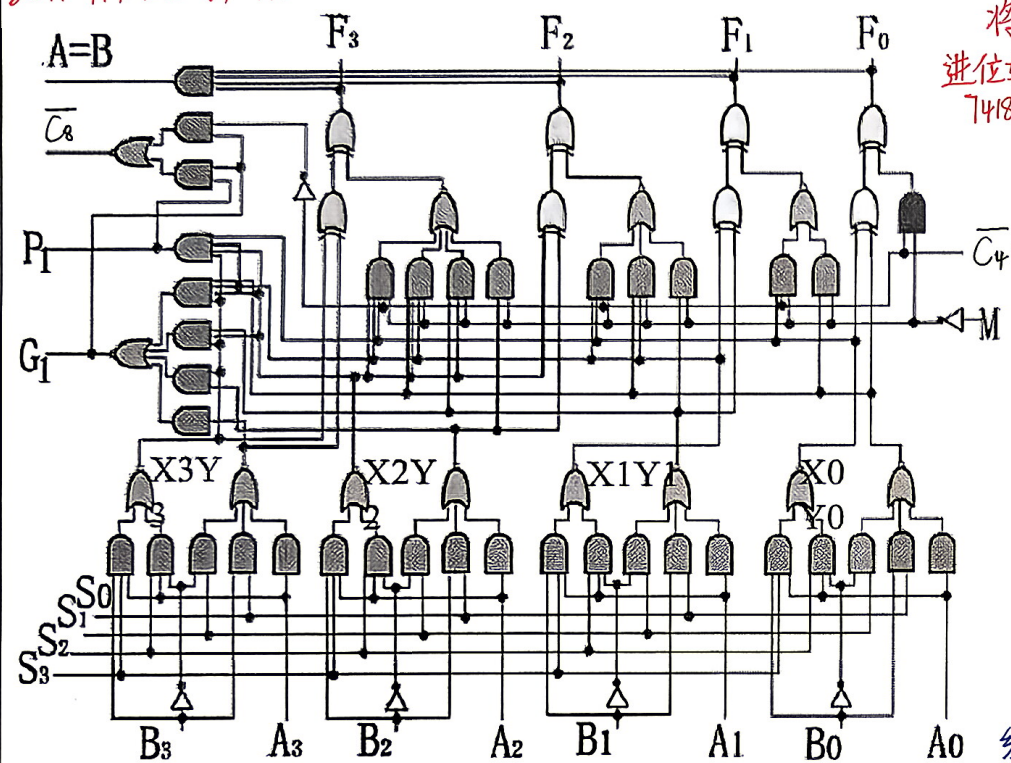
组与组串行连接

组与组并行连接

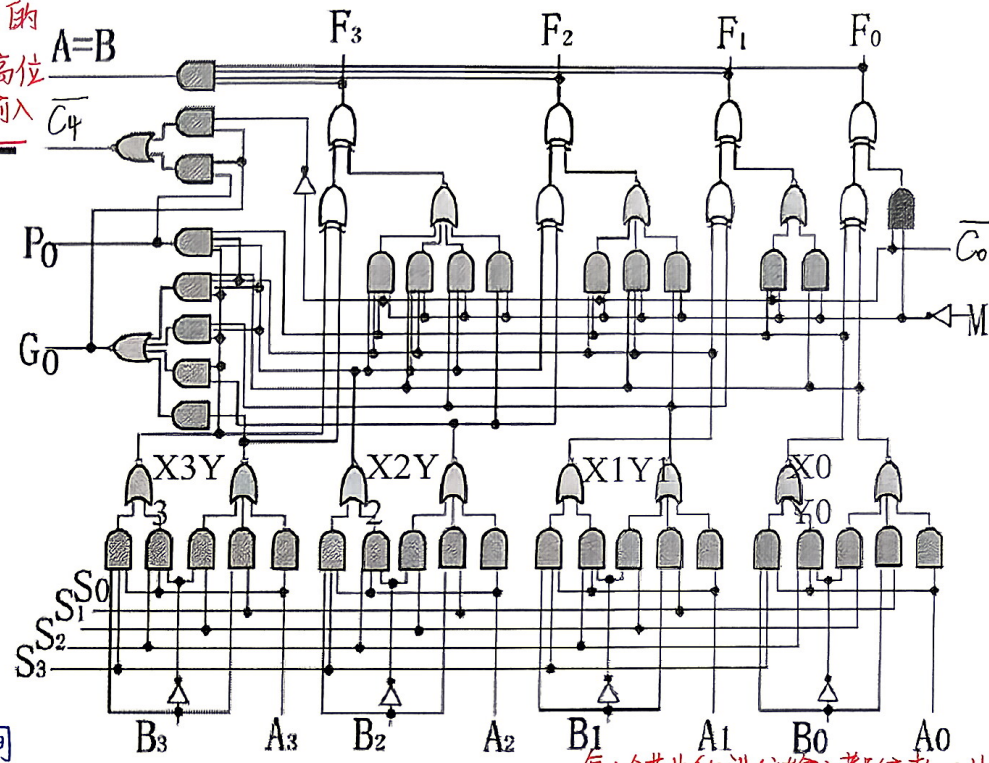
区别: 进位信号传递不同

① 组间串行进位

还有2片在左边,共4片



将低位74181的进位输出作为高位74181的进位输入



进位关系

组内

先行进位

$$C_{n+1} = Y_0 + X_0 C_n$$

$$C_{n+2} = Y_1 + X_1 C_{n+1} = Y_1 + Y_0 X_1 + X_0 X_1 C_n$$

$$C_{n+3} = Y_2 + X_2 C_{n+2} = Y_2 + Y_1 X_1 + Y_0 X_1 X_2 + X_0 X_1 X_2 C_n$$

$$C_{n+4} = Y_3 + X_3 C_{n+3} = Y_3 + Y_2 X_3 + Y_1 X_2 X_3 + Y_0 X_1 X_2 X_3 + X_0 X_1 X_2 X_3 C_n$$

$$G = Y_3 + Y_2 X_3 + Y_1 X_2 X_3 + Y_0 X_1 X_2 X_3$$

$$P = X_0 X_1 X_2 X_3$$

$$C_1 = Y_0 + C_0 X_0$$

$$C_2 = Y_1 + C_1 X_1$$

$$C_3 = Y_2 + C_2 X_2$$

组内 (串) 组间 (并)

$$C_4 = G_0 + C_0 P_0$$

$$C_8 = G_1 + C_4 P_1$$

$$C_{12} = G_2 + C_8 P_2$$

$$C_{16} = G_3 + C_{12} P_3$$

每个芯片的进位输入都依赖于其右边芯片的进位输出

组间

串行进位

$$C_4 = G_0 + P_0 C_0$$

$$C_8 = G_1 + P_1 C_4$$

$$C_{12} = G_2 + P_2 C_8$$

$$C_{16} = G_3 + P_3 C_{12}$$

与进位无关,仅芯片内部x和y的函数

② 组间并行进位 (两级先行进位的ALU)

由串行进位关系

$$\begin{cases} C_4 = G_0 + P_0 C_0 \\ C_8 = G_1 + P_1 C_4 \\ C_{12} = G_2 + P_2 C_8 \\ C_{16} = G_3 + P_3 C_{12} \end{cases}$$

得: $C_4 = G_0 + P_0 C_0$

$$C_8 = G_1 + P_1 C_4 = G_1 + P_1 (G_0 + P_0 C_0) = G_1 + G_0 P_1 + P_0 P_1 C_0$$

$$C_{12} = G_2 + P_2 C_8 = G_2 + P_2 (G_1 + G_0 P_1 + P_0 P_1 C_0) = G_2 + G_1 P_2 + G_0 P_1 P_2 + P_0 P_1 P_2 C_0$$

$$C_{16} = G_3 + P_3 C_{12} = G_3 + P_3 (G_2 + G_1 P_2 + G_0 P_1 P_2 + P_0 P_1 P_2 C_0) = G_3 + G_2 P_3 + G_1 P_2 P_3 + G_0 P_1 P_2 P_3 + P_0 P_1 P_2 P_3 C_0$$

最高的进位输出与中间的进位无关.

只与所有的 G_i, P_i 有关.

$$= G^* + P^* C_0$$

仅与 X_i, Y_i 有关, 与芯片有关.

再加块电路, 把这些逻辑单独实现, 让 C_4, C_8, C_{12}, C_{16} 不用等, 同时产生.

进位

74182 (先行进位部件)

∴ 所有的 G_i, P_i 与进位无关, 16 位的数给到 4 个芯片, 它们的 G_i, P_i 同时出来,

再把它送到加的电路中, 那 C_4, C_8, C_{12}, C_{16} 的输入同时都有了, 即可同时产生.

再经4个74181产生C16

成组进位发生输出

成组进位传递输出

经10T产生进位输入 C_4, C_8, C_{12}

外部

$$P^* = P_0 P_1 P_2 P_3$$

$$G^* = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0$$

74182 先行进位部件(CLA)
(并)

共14T

再经4

C16

整个电路最高位的进位输出是在最左边的74181上

内部

C_0 既作最低位的进位输入

整个16位的进位

又作整个先行部件的输入

经T,可产生所有的P和G, 74182的输入就有3

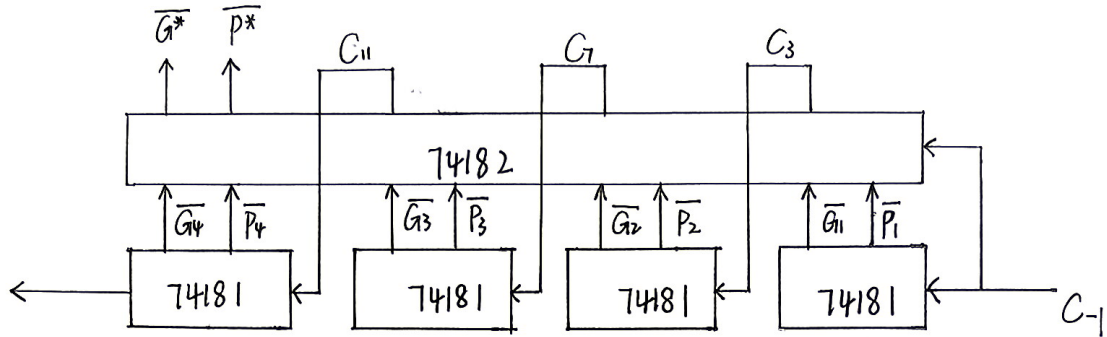
计算机组成与体系结构

16位的数分别接在4个74181上,最低位的进位接在最右边74181的进位输入,另一边接输出上。

16位字长ALU的构成

74181 ALU 设置3 P、G 两个本组先行进位输出端。

若将4片74181的P、G 输出端送入到74182 先行进位部件(CLA), 又可实现第2级的先行进位, 即组与组之间的先行进位。

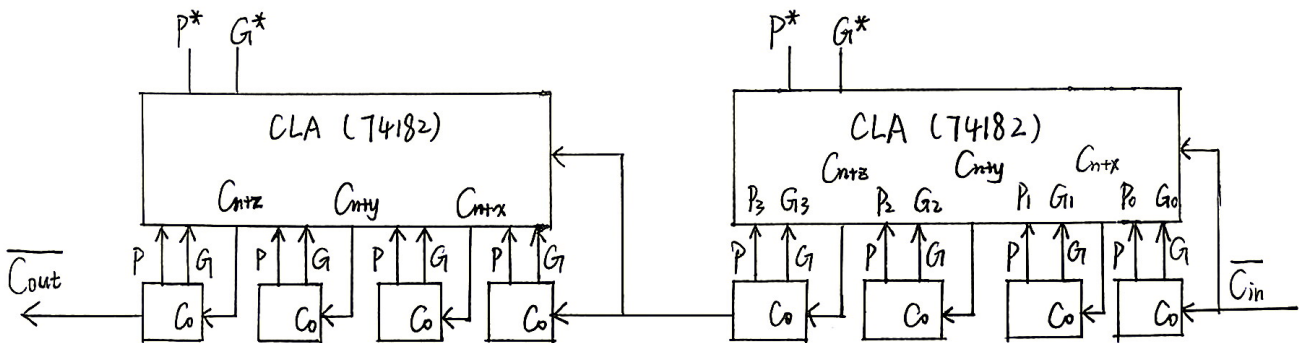


最低位进位接到最右边74181上,
同时接到先行进位最右边的
进位输入上。

32位字长的ALU的构成

若计算机是32位, 那么就是“全字长”。

用若干个74181 ALU 位片, 与配套的74182 先行进位部件CLA在一起, 可构成一个全字长ALU。



64位字长的ALU呢? (用74181和74182)

① 串接ALU, 将当前32位的再用1个32位的接上, 速度较慢 (大组与大组串行)

② 用4大组16位的ALU, 再加上1个74182, 构成一个大组与大组之间先行进位的64位ALU
(并)

浮点运算方法和浮点运算器

1) 浮点加、减法运算

设有两个浮点数 x 和 y , 它们分别为: $x = 2^{E_x} \cdot M_x$
 $y = 2^{E_y} \cdot M_y$

其中, E_x 和 E_y 分别是数 x 和 y 的阶码, M_x 和 M_y 分别是数 x 和 y 的尾数

两浮点数进行加法和减法的运算规则是:

$$x \pm y = (M_x 2^{E_x - E_y} \pm M_y) 2^{E_y} \quad (E_x \leq E_y)$$

完成浮点加减运算的操作过程大体有 6 步:

① 0 操作数的检查

即小数点对齐

② 比较阶码大小并完成对阶

→ 小阶向大阶看齐, 阶码小的化成与阶码大的相同且尾数右移

然后就方便尾数进行加/减

阶码变大

③ 尾数进行加/减运算

尾数都属于定点小数的形式, 一定要小阶向大阶看齐

④ 结果规格化 (保证精度最高)

⑤ 舍入处理 (若规格化后尾数位数过长, 就去掉一些)

⑥ 溢出处理 (上面步骤修改了阶, 我们要判断阶是否超出所能表示的最大的阶)

对阶:

使两数阶码相同 (小数点位置对齐)

先求两数阶码之差 $\Delta E = E_x - E_y$

若 $\Delta E = 0$, 则 $E_x = E_y$

若 $\Delta E > 0$, 则 $E_x > E_y$

若 $\Delta E < 0$, 则 $E_x < E_y$

通过移动尾数来改变 E_x 或 E_y , 使之相等。

原则:

阶码小的数向阶码大的数对齐。

小阶尾数右移, 每右移 1 位, 其阶码加 1. (右规)

例. $x = 2^0 \times 0.1101$, $y = 2^{-1} \times (-0.1010)$, 求 $x+y = ?$

解:

为便于直观地了解, 将它们用浮点数形式表示, 两数均以补码表示, 阶码和尾数均采用双符号位.

(普通的表达式, 非 IEEE 标准)

$$[x]_{\text{补}} = 00.01, 00.1101 \quad [y]_{\text{补}} = 00.11, 11.0110$$

$$[\Delta E]_{\text{补}} = [E_x]_{\text{补}} - [E_y]_{\text{补}} = 00.01 - 00.11 = 00.01 + 11.01 = 11.10$$

$\Delta E = -2$, 说明 E_x 比 E_y 小 2, 故将 x 的尾数右移 2 位

补码移位 $\left\{ \begin{array}{l} \text{正数: 左边添0} \\ \text{负数: 左边添1} \end{array} \right.$

先右移 1 位, 得 $[x]_{\text{补}} = 0010, 00.0110$

再右移 1 位, 得 $[x]_{\text{补}} = 0011, 00.0011$

至此, $\Delta E = 0$, 对齐完毕

尾数求和方法与定点加减法运算完全一样.

对阶完毕可得:

$$[x]_{\text{补}} = 0011, 00.0011$$

$$[y]_{\text{补}} = 0011, 11.0110$$

对尾数求和:

$$\begin{array}{r} 00.0011 \\ + 11.0110 \\ \hline 01.1001 \end{array}$$

即得: $[x+y]_{\text{补}} = 0011, 11.1001$

符号表示的尾数, 不是规格化形式!

注意: 浮点数运算过程中尾数溢出并非真的溢出, 尾数溢出要通过规格化去修改它的阶

若阶溢出, 才是真正的溢出.

结果规格化

求和之后得到的数可能不是规格化了的数, 为增加有效数字的位数, 提高运算精度, 必须将求和结果规格化.

① 规格化的定义: $\frac{1}{2} \leq |S| < 1$ (二进制)

运算结果的尾数

不论正或负, 用原码表示, 小数点后第 1 位必须是 1

采用原码:

正数: $S = 0.1xxx...x$

负数: $S = 1.1xxx...x$

采用双符号位的补码:

正数: $S = 00.1xxx...x$

负数: $S = 11.0xxx...x$

② 向左规格化

若非规格化的数, 需要尾数向左移位, 以实现规格化的过程, 称之为 向左规格化

0011, 11.1001 不是规格化数, 需要左规, 即:
 若 00.0010xx..., 也说明绝对值太小, 需要左规
 左移1位, 阶码减1.

得: $[x+y]_{补} = 0010, 11.0010$

③ 向右规格化

浮点加减运算时, 尾数求和的结果也可能得到: 01.xxx...x 或 10.xxx...x.
 双符号位, 最高符号位是正确的符号. 这是个正数且溢出了. 说明尾数的绝对值大于1了.

即两符号位不等, 即结果的绝对值大于1. 向左破坏了规格化. 这是个负数, 而且溢出了, 说明绝对值超出1了.

此时将尾数运算的结果右移1位, 阶码加1, 称之为 向右规格化

例. 两浮点数 $x = 0.1101 \times 2^{10}$, $y = (0.1011) \times 2^{01}$, 求 $x+y$.

解: $[x]_{补} = 0010, 00.1101$ $[y]_{补} = 0001, 00.1011$

对阶: $[\Delta E]_{补} = [E_x]_{补} - [E_y]_{补} = 0010 - 0001 = 0010 + 1111 = 0001 > 0$

$\therefore [E_y]_{补} < [E_x]_{补}$

因此, y向x对齐, 将y的尾数右移1位, 阶码加1.

$[y]_{补} = 0010, 00.0101$ 至此, $\Delta E = 0$, 对阶完毕

求和:

$$\begin{array}{r} 00.1101 \\ + 00.0101 \\ \hline 01.0010 \end{array}$$

$[x+y]_{补} = 0010, 01.0010$

规格化:

$\therefore$ 运算结果的两符号位不同, 其绝对值大于1, 因此右规.

$\therefore [x+y]_{补} = 0011, 00.1001$

舍入处理

在对阶或向右规格化时,尾数要向右移位,就会导致被右移的尾数的低位部分会丢失,造成一定的误差,因此需进行舍入处理.

简单的舍入方法有 2 种:

① "0舍1入"法

若右移时被丢掉数位的最高位是 0 则舍去,反之则将尾数的末位加 "1".

② "恒置 1" 法

只要数位被移掉,就在尾数的末位恒置 "1".

另, IEEE 754 标准中,舍入处理提供了 4 种可选方法.

溢出处理

与定点加减法一样,浮点加减运算最后一步也需判溢出.

在浮点规格化中已指出,当尾数之和(差)出现 $01.x \dots x$ 或 $10.x \dots x$ 时并不表示溢出,只有将该数右规后,再根据阶码来判断浮点运算结果是否溢出.

★ 尾数'溢出' $\rightarrow$ 规格化 $\rightarrow$ (可能的舍入处理) $\rightarrow$ 依调整后的阶码判溢出

小结:

浮点数的溢出是与其阶码溢出表现出来的.

在加减运算过程中要检查是否产生了溢出:

若阶码正常,加减运算正常结束;

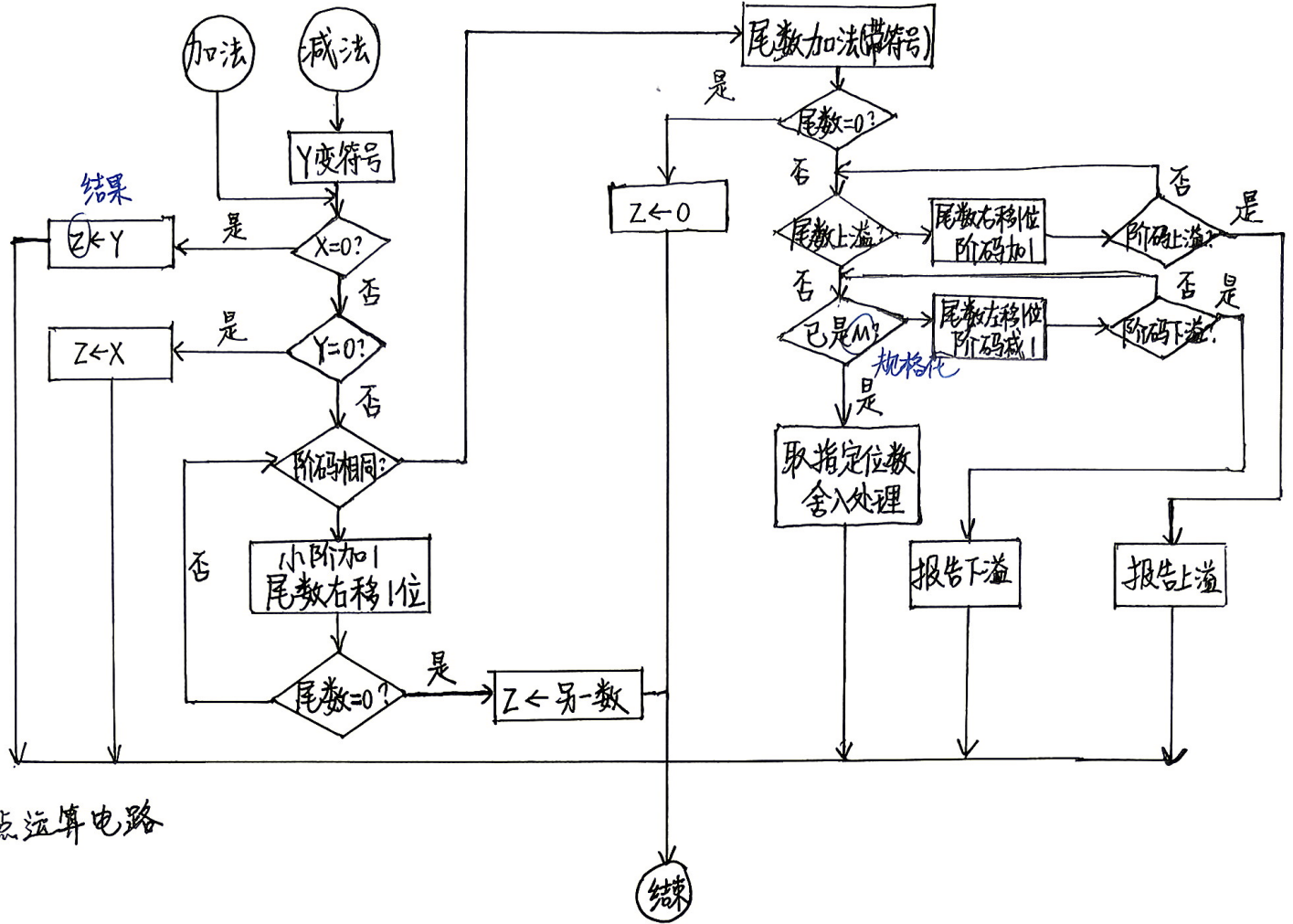
若阶码溢出,进行相应处理.

{ 阶码上溢: 超过阶码可能表示的最大值的正指数值,可认为是 $+\infty$ 和 $-\infty$.
阶码下溢: 超过阶码可能表示的最小值的负指数值,可认为是 0.

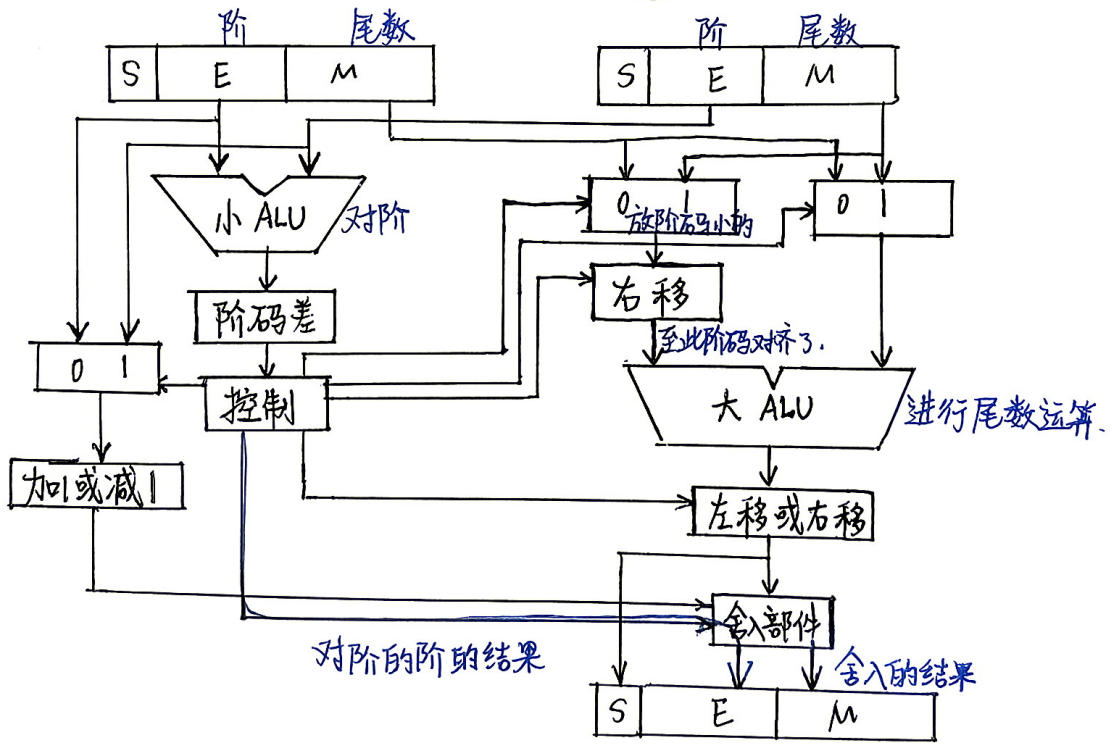
另,

{ 尾数上溢: 右规
尾数下溢: 左规

流程图



浮点运算电路



2) 浮点乘法、除法运算

设有两个浮点数 x 和 y :

$$x = 2^{E_x} \cdot M_x$$
$$y = 2^{E_y} \cdot M_y$$

浮点乘法运算规则: $x \times y = 2^{\frac{(E_x + E_y)}{2}} \cdot (M_x \times M_y)$

阶码: 两阶码之和
尾数: 两尾数之积

浮点除法运算规则: $x \div y = 2^{\frac{(E_x - E_y)}{2}} \cdot (M_x \div M_y)$

阶码: 两阶码之差
尾数: 两尾数之商

浮点乘、除运算步骤:

- ① 0 操作数检查
- ② 阶码加/减操作 (定点整数加、减法)
- ③ 尾数乘/除操作 (定点小数乘、除法)
- ④ 结果规格化及舍入处理

浮点数的阶码运算

- 对阶码的运算有 +1、-1、两阶码求和、两阶码求差 4 种, 运算时必须检查结果是否溢出 (是否超出阶码的表示范围)
- 在计算机中, 阶码通常用补码或移码的形式来表示。

移码运算规则和判定溢出的方法。

移码的定义为 $[x]_{\text{移}} = 2^n + x \quad (-2^n \leq x < 2^n)$

按此定义, 则有 $[x]_{\text{移}} + [y]_{\text{移}} = 2^n + x + 2^n + y = 2^n + (2^n + (x+y)) = 2^n + [x+y]_{\text{移}}$

$$\therefore [x+y]_{\text{移}} = -2^n + [x]_{\text{移}} + [y]_{\text{移}}$$

混合使用移码和补码

考虑到移码和补码的关系:

对同一个数值, 其数值位完全相同, 而符号位正好完全相反。

$[y]_{\text{补}}$ 的定义是: $[y]_{\text{补}} = 2^{n+1} + y$ (除符号位外的长度)

求阶码和可用如下方式完成:

$$[x]_{\text{移}} + [y]_{\text{补}} = 2^n + x + 2^{n+1} + y = 2^{n+1} + (2^n + (x+y))$$

即: $[x+y]_{\text{移}} = [x]_{\text{移}} + [y]_{\text{补}} \pmod{2^{n+1}}$

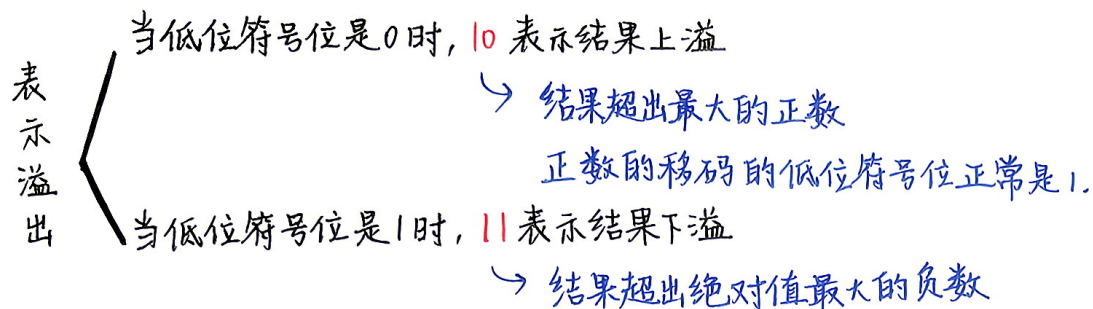
$$[x-y]_{\text{移}} = [x]_{\text{移}} + [-y]_{\text{补}} \pmod{2^{n+1}}$$

混合运算的阶码运算结果溢出处理

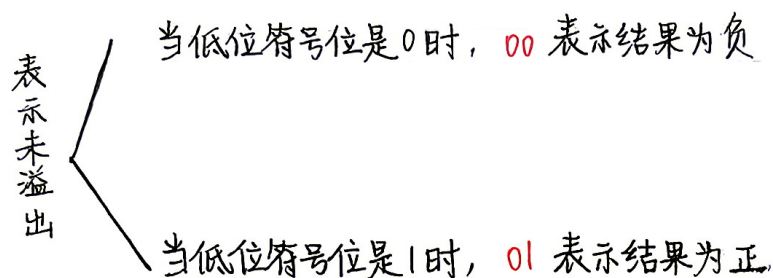
使用双符号位的阶码加法器, 并规定移码的第2个符号位,

即最高符号位恒用0参加加减运算, 故溢出条件是结果的最高符号位是1

最高符号位是1



最高符号位是0



例. $x = +011$, $y = +110$, 求 $[x+y]_{\text{移}}$ 和 $[x-y]_{\text{移}}$, 并判断是否溢出

解: 阶码取3位(不含符号位), 其对应的真值范围是 $-8 \sim +7$

$$\therefore [x]_{\text{移}} = 01011, [y]_{\text{补}} = 00110, [-y]_{\text{补}} = 11010$$

$$\therefore [x+y]_{\text{移}} = [x]_{\text{移}} + [y]_{\text{补}} = \begin{array}{r} 01011 \\ +00110 \\ \hline 10001 \end{array} \rightarrow \text{结果上溢}$$

$$[x-y]_{\text{移}} = [x]_{\text{移}} + [-y]_{\text{补}} = \begin{array}{r} 01011 \\ +11010 \\ \hline 00101 \end{array} \rightarrow \text{结果正确是}-3$$

减去偏移量8, 结果确实是-3.

尾数处理

浮点加减法对结果的规格化及舍入处理亦适用于浮点乘法

方法一:

无条件地丢掉正常尾数最低位之后的全部数值。

这种办法被称为截断处理，优点是处理简单，缺点是影响结果的精度

方法二:

运算过程中保留右移中移出的若干高位的值，最后再按某种规则用这些位上的值修正尾数。这是舍入处理

舍入处理:

当尾数用原码表示时

第1种: 只要尾数的最低位是1, 或移出的几位中有是1的数值位, 就令最低位的值是1。

第2种: 当丢失的最高位的值是1, 把这个1加到最低数值位上进行修正, 否则, 直接舍去丢失的各位的值。

这样处理时, 舍入效果对正数、负数相同

"0舍1入"法

↓ ↓
使数的 使数的
绝对值 绝对值
变小 变大

当尾数用补码表示时

采用0舍1入法时, 若丢失的位不全是0。

对正数来讲, 舍入的结果与原码分析相同

对负数来讲, 舍入的结果与原码分析相反(即"舍"使绝对值变大, "入"使绝对值变小), 为了使原、补码舍入处理后的结果相同, 对负数可采用如下规则进行舍入处理:

当丢失的各位都是0时, 不必舍入;

当丢失的最高位是0, 以下各位不全为0, 或丢失的最高位是1, 以下各位都是0时, 舍去丢失位上的值;

当丢失的最高位是1, 以下各位不全为0, 就执行在尾数最低位入1的修正操作。

例. 设 $[x_1]_{补} = 11.01100000$, $[x_2]_{补} = 11.01100001$, $[x_3]_{补} = 11.01101000$, $[x_4]_{补} = 11.01111001$,

求执行只保留小数点后4位有效数字的舍入操作值.

解: 执行舍入操作后, 结果值分别是:

$$[x_1]_{补} = 11.0110$$

$$[x_2]_{补} = 11.0110$$

$$[x_3]_{补} = 11.0110$$

$$[x_4]_{补} = 11.1000$$

例. 设有浮点数 $x = 2^{(5)} \times 0.0110011$, $y = 2^3 \times (-0.1110010)$, 阶码用4位移码表示, 尾数(含符号位)用8位补码表示. 用 $[x \times y]_{浮}$. (要求: 用补码完成尾数乘法运算, 运算结果尾数保留高8位(含符号位), 并用尾数低位字长值处理舍入操作).

偏移量是其一半. $2^3 = 8$

解:

移码采用双符号位, 尾数补码采用单符号位, 则有

$$[M_x]_{补} = 0.0110011 \quad [M_y]_{补} = 1.0001110$$

$$[E_x]_{移} = 00(01) \quad [E_y]_{移} = 01011 \quad [E_y]_{补} = 00011$$

$$[x]_{浮} = \underbrace{00011}_{移}, \underbrace{0.0110011}_{补} \quad [y]_{浮} = \underbrace{01011}_{移}, \underbrace{1.0001110}_{补}$$

① 求阶码和:

$$\therefore [E_x + E_y]_{移} = [E_x]_{移} + [E_y]_{补} = 00011 + 00011 = 00110$$

$\therefore$ 真实的值是 -2

② 尾数乘法运算 (可采用补码阵列乘法器实现):

$$[M_x]_{补} \times [M_y]_{补} = [0.0110011]_{补} \times [1.0001110]_{补}$$

$$= [1.1010010, 1001010]_{补}$$

符号位与最高数值位相同, 不是规格化形式.

③ 对尾数进行规格化处理

上面所得乘积的尾数符号位与最高数值位符号相同, 不是规格化数, 需左规, 阶码减1, 变为 $00101(-3)$, 尾数变为 1.01001010010100

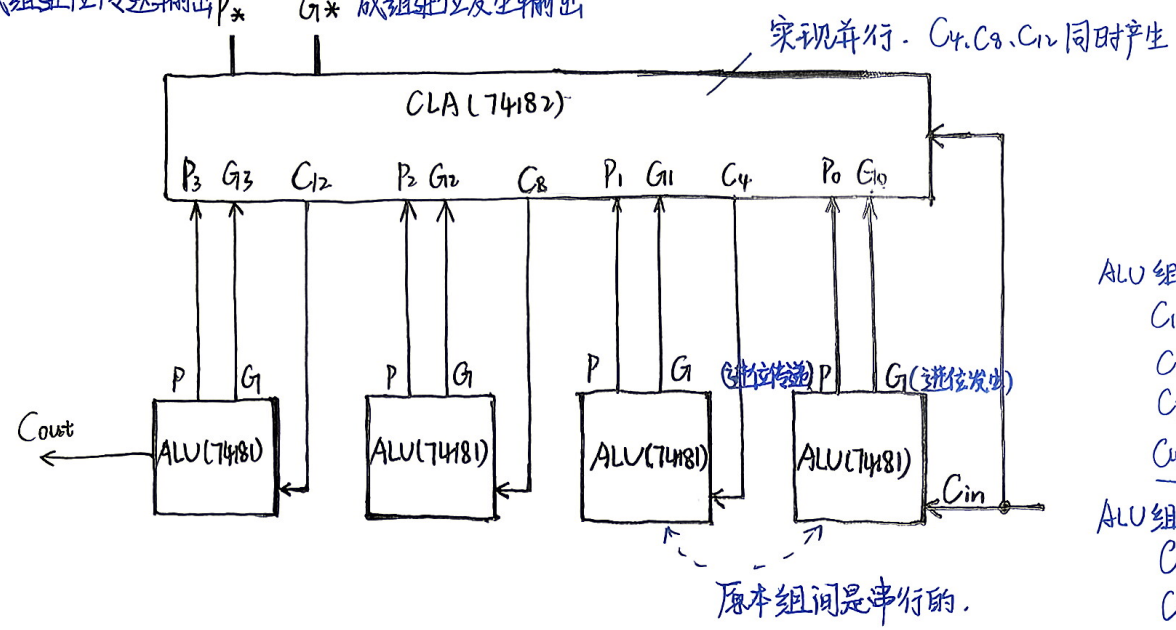
④ 舍入处理

尾数是负数, 取尾数高位字长, 按照舍入规则, 舍去低位字长, 得尾数 1.0100101 .

$$\text{最终, 相乘结果是 } [x \times y]_{浮} = 00101, 1.0100101$$

$$\text{真值: } x \times y = 2^{-3} \times (-0.1011011)$$

16位先行进位部件
成组进位传递输出 P^*, G^* 成组进位发生输出

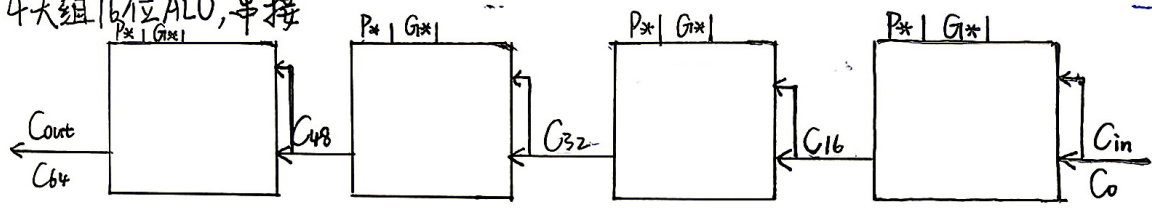


ALU 组内(先行):
 $C_1 = y_0 + C_0 x_0$
 $C_2 = y_1 + C_1 x_1$
 $C_3 = y_2 + C_2 x_2$
 $C_4 = y_3 + C_3 x_3 \rightarrow$ 产生 P, G
 ALU 组间:
 $C_4 = G_0 + C_0 P_0$
 $C_8 = G_1 + C_4 P_1$
 $C_{12} = G_2 + C_8 P_2$
 $C_{16} = G_3 + C_{12} P_3 \rightarrow$ 产生 P^*, G^*
 就能实现组间并行

64位?

① 4大组16位ALU, 串接

组间串行



② 4大组16位ALU + 1个74182 (组与组间, 成组的) 先行进位

